

Process Concepts

2520030030

U.Rohit Sai

Programm - 1

Code:

```
rohitsai@MSI: ~/OS
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main() {
printf("Process ID: %d\n", getpid());
printf("Parent Process ID: %d\n", getppid());
return 0;
}
```

Output:

```
rohitsai@MSI: ~/OS
rohitsai@MSI:~$ cd OS
rohitsai@MSI:~/OS$ nano process.c
rohitsai@MSI:~/OS$ gcc process.c -o process
rohitsai@MSI:~/OS$ ./process
Process ID: 538
Parent Process ID: 318
```

Programm - 2

Code:

```
rohitsai@MSI: ~/OS
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        printf("Fork failed.\n");
    }
    else if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
    }
    else {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
    }
    return 0;
}
```

Output:

```
rohitsai@MSI:~/OS$ nano Fork.c
rohitsai@MSI:~/OS$ gcc Fork.c -o Fork
rohitsai@MSI:~/OS$ ./Fork
Parent Process
Parent PID: 613
Child Process
Child PID: 614
```

Programm - 3

Code:

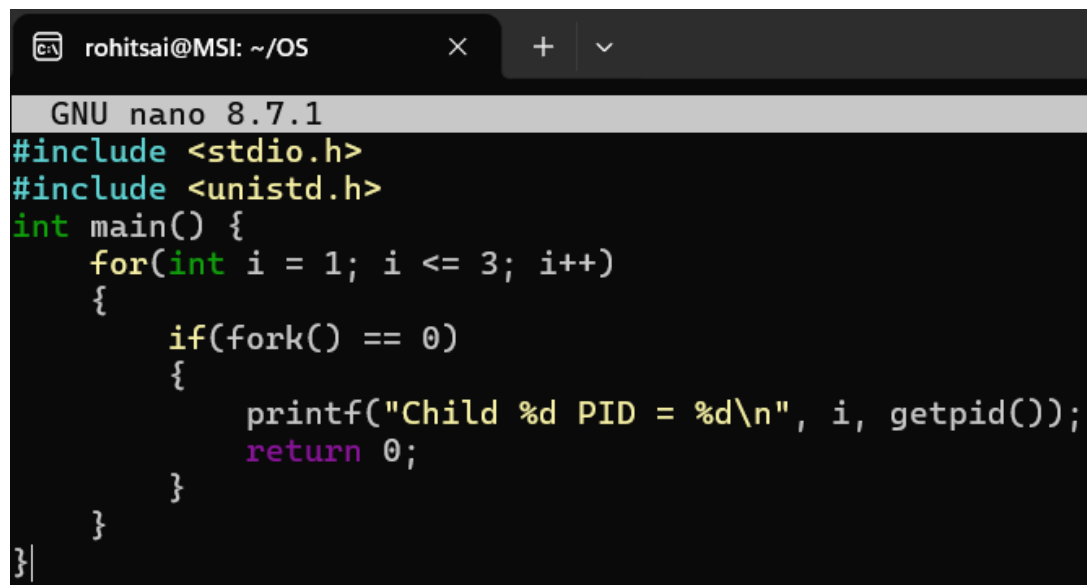
```
rohitsai@MSI: ~/OS
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        printf("Fork failed.\n");
    }
    else if (pid == 0) {
        printf("Child process is running...\n");
        sleep(2);
        printf("Child process finished.\n");
    }
    else {
        printf("Parent is waiting for child...\n");
        wait(NULL);
        printf("Parent continues after child finishes.\n");
    }
    return 0;
}
```

Output:

```
rohitsai@MSI:~/OS$ nano Wait.c
rohitsai@MSI:~/OS$ gcc Wait.c -o Wait
rohitsai@MSI:~/OS$ ./Wait
Parent is waiting for child...
Child process is running...
Child process finished.
Parent continues after child finishes.
```

Programm - 4

Code:



```
rohitsai@MSI: ~/OS
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main() {
    for(int i = 1; i <= 3; i++)
    {
        if(fork() == 0)
        {
            printf("Child %d PID = %d\n", i, getpid());
            return 0;
        }
    }
}
```

Output:

```
rohitsai@MSI:~/OS$ nano Multiplechild.c
rohitsai@MSI:~/OS$ gcc Multiplechild.c -o Multiplechild
rohitsai@MSI:~/OS$ ./Multiplechild
Child 1 PID = 722
Child 2 PID = 723
Child 3 PID = 724
```

Programm - 5

Code:

```
rohitsai@MSI: ~/OS
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main()
{
    printf("Before exec()\n");
    execlp("ls", "ls", "-l", NULL);
    printf("This will not execute.\n");
    return 0;
}
```

Output:

```
rohitsai@MSI:~/OS$ nano execlp.c
rohitsai@MSI:~/OS$ gcc execlp.c -o execlp
rohitsai@MSI:~/OS$ ./execlp
Before exec()
total 188
-rwxr-xr-x 1 rohitsai rohitsai 16168 Aug 12 06:44 Copy
-rw-r--r-- 1 rohitsai rohitsai  440 Aug 12 06:43 Copy.c
-rwxr-xr-x 1 rohitsai rohitsai 16080 Aug 16 04:03 Fork
-rw-r--r-- 1 rohitsai rohitsai  361 Aug 16 04:01 Fork.c
-rwxr-xr-x 1 rohitsai rohitsai 16304 Aug 12 06:31 Linux
-rw-r--r-- 1 rohitsai rohitsai  556 Aug 12 06:36 Linux.c
-rw-r--r-- 1 rohitsai rohitsai  220 Aug 16 04:08 MultipleChild
-rwxr-xr-x 1 rohitsai rohitsai 16048 Aug 16 04:10 Multiplechild
-rw-r--r-- 1 rohitsai rohitsai  220 Aug 16 04:09 Multiplechild.c
-rw-r--r-- 1 rohitsai rohitsai  423 Aug  6 04:02 Practical_2
-rw-r--r-- 1 rohitsai rohitsai  439 Aug  6 04:17 Practical_2.c
-rwxr-xr-x 1 rohitsai rohitsai 16312 Aug  5 08:27 Pratical_1
-rw-r--r-- 1 rohitsai rohitsai  440 Aug  5 08:27 Pratical_1.c
-rwxr-xr-x 1 rohitsai rohitsai 16072 Aug 16 04:06 Wait
-rw-r--r-- 1 rohitsai rohitsai  465 Aug 16 04:06 Wait.c
-rw-r--r-- 1 rohitsai rohitsai  12 Jul 29 06:03 def.txt
-rwxr-xr-x 1 rohitsai rohitsai 16000 Aug 16 04:11 execlp
-rw-r--r-- 1 rohitsai rohitsai  175 Aug 16 04:11 execlp.c
-rw-r--r-- 1 rohitsai rohitsai  12 Aug 12 06:41 input.txt
-rw-r--r-- 1 rohitsai rohitsai  12 Aug 12 06:44 output.txt
-rwxr-xr-x 1 rohitsai rohitsai 16040 Aug 16 04:00 process
-rw-r--r-- 1 rohitsai rohitsai  148 Aug 16 04:00 process.c
-rw-r--r-- 1 rohitsai rohitsai  23 Jul 29 08:13 programm.txt
```

Programm - 6

Code:

```
rohitsai@MSI: ~/OS
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main()
{
    fork();
    for(int i = 1; i <= 5; i++)
    {
        printf("PID: %d Count: %d\n", getpid(), i);
        sleep(1);
    }
    return 0;
}
```

Output:

```
rohitsai@MSI:~/OS$ nano concurrently.c
rohitsai@MSI:~/OS$ gcc concurrently.c -o concurrently
rohitsai@MSI:~/OS$ ./concurrently
PID: 795 Count: 1
PID: 796 Count: 1
PID: 795 Count: 2
PID: 796 Count: 2
PID: 795 Count: 3
PID: 796 Count: 3
PID: 795 Count: 4
PID: 796 Count: 4
PID: 795 Count: 5
PID: 796 Count: 5
```

Programm - 7

Code:

```
rohitsai@MSI: ~/OS
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Exiting\n");
        return 0;
    }
    else
    {
        sleep(15);
        printf("Parent Finished\n");
    }

    return 0;
}
```

Output:

```
rohitsai@MSI:~/OS$ nano zombie.c
rohitsai@MSI:~/OS$ gcc zombie.c -o zombie
rohitsai@MSI:~/OS$ ./zombie
Child Exiting
Parent Finished
```

Programm - 8

Code:

```
rohitsai@MSI: ~/OS
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid = fork();
    if(pid > 0)
    {
        printf("Parent Exiting\n");
        return 0;
    }
    else
    {
        sleep(5);
        printf("Child PID : %d\n", getpid());
        printf("New Parent PID : %d\n", getppid());
    }
    return 0;
}
```

Output:

```
rohitsai@MSI:~/OS$ nano orphan.c
rohitsai@MSI:~/OS$ gcc orphan.c -o orphan
rohitsai@MSI:~/OS$ ./orphan
Parent Exiting
rohitsai@MSI:~/OS$ Child PID : 870
New Parent PID : 317
```